

**Smt. Chandaben Mohanbhai Patel Institute of Computer Applications,
CHARUSAT**

**BCA Semester-III
CAUE302: Python Programming**

Week 4: Sequence Types Supported by Python (Strings, Lists, and Tuples)

1. Introduction to Sequence Types

- A Sequence in Python is an ordered collection of items where each element is stored at a specific integer position known as an index.
- Index positioning in Python is zero-based, meaning the very first element resides at index 0.
- Python natively supports three foundational sequence types:
Strings (text sequences), Lists (mutable ordered collections), and Tuples (immutable ordered collections).

Graphical Idea: What Exactly Is a "Sequence"?

Picture a sequence as a row of numbered boxes placed next to each other. Every box holds exactly one item, and every box has a position number (its index) written under it, starting from 0 on the left. Python also lets you count backwards from the right end using negative numbers, where -1 is always the last box.

Index →	0	1	2	3	4
Item	a	b	c	d	e
Negative Index →	-5	-4	-3	-2	-1

This one picture — numbered boxes holding items in order — is exactly what a String, a List and a Tuple all look like underneath. The only difference between the three is what kind of items sit inside the boxes, and whether the boxes can be edited afterwards. The table below shows this side by side.


```
print('Last Character (index -1):', word[-1])
print('Second Last Character (index -2):', word[-2])
```

Complete Example 2: String Slicing and Reversing:

```
text = 'CHARUSAT UNIVERSITY'
print('Slice from index 0 to 8:', text[0:8])
print('Slice from index 9 to end:', text[9:])
print('Every 2nd character:', text[::2])
print('Reversed String:', text[::-1])
```

Graphical Representation: Indexing on word = 'PYTHON'

Every character of the string sits inside its own numbered box, exactly like the sequence diagram above. `word[0]` simply means "look inside the box under positive index 0", and `word[-1]` means "look inside the box under negative index -1" (the rightmost box).

Positive Index	0	1	2	3	4	5
Character	P	Y	T	H	O	N
Negative Index	-6	-5	-4	-3	-2	-1

Graphical Representation: Slicing on text = 'CHARUSAT UNIVERSITY'

Slicing `text[start:stop]` grabs every box from 'start' up to (but not including) 'stop'. The blue boxes below are what `text[0:8]` returns ('CHARUSAT'); the green boxes are what `text[9:]` returns ('UNIVERSITY'); the small `␣` box in between is the untouched space character.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C	H	A	R	U	S	A	T	␣	U	N	I	V	E	R	S	I	T	Y

Topic 2: Essential Built-In String Methods

Python provides a large collection of string methods to transform cases, perform searches, sanitize input whitespace, split strings into lists, and join lists into formatted text strings.

String (str) Methods:

- Case Transformations: upper(), lower(), title(), capitalize(), swapcase()
- Searching & Counting: find(), count(), startswith(), endswith()
- Sanitization & Modification: strip(), replace()
- Character Validation Checks: isalpha(), isdigit(), isalnum(), isspace(), isupper(), islower()

```
msg = 'welcome to python programming'

print('Uppercase:', msg.upper())

print('Title Case:', msg.title())

print('Count of letter o:', msg.count('o'))

print('Find index of python:', msg.find('python'))

print('Starts with welcome?:', msg.startswith('welcome'))

code = 'BCA2026'

print('Is code alphanumeric?:', code.isalnum())

print('Is code numeric?:', code.isdigit())
```

Example 1: Case Conversion & Whitespace Stripping:

```
user_input = ' Hello World! '

print('Uppercase:', user_input.upper())
print('Lowercase:', user_input.lower())
print('Stripped Whitespace:', user_input.strip())
print('Title Case:', 'python programming'.title())
```

Graphical Working of String Methods

A string method behaves like a small machine: the original string goes in on the left, the method does its job in the middle, and a brand-new string (or list) comes out on the right.

The box on the left never changes -- strings are immutable, so the method always hands back a fresh result.

Input: 'hello'	.upper()	Output (new string): 'HELLO'
----------------	-----------------	------------------------------

The same before/after idea applies to every method below:

Method	Before (original string)	After (new value produced)
.upper()	'welcome to python'	'WELCOME TO PYTHON'
.strip()	' Hello World! '	'Hello World!'
.split(',')	'apple,banana,mango'	['apple', 'banana', 'mango']
.replace('apple','pineapple')	'apple,banana'	'pineapple,banana'

3. Part B: Lists (list)

A List is a mutable, ordered collection of heterogeneous elements enclosed within square brackets `[]`. Because lists are mutable, their contents can be modified in place—elements can be appended, inserted, replaced, removed, or reordered without changing the memory identity of the list object.

List (list) Methods Added:

- Adding / Inserting / Merging: `append()`, `insert()`, `extend()`
- Removing / Clearing: `remove()`, `pop()`, `clear()`
- Searching & Counting: `index()`, `count()`
- Sorting & Ordering: `sort()`, `sort(reverse=True)`, `reverse()`
- Copying & Aggregation: `copy()`, `len()`, `sum()`, `min()`, `max()`

Topic 3: List Creation & Dynamic Modifications (`append`, `insert`, `extend`, `remove`, `pop`)

Elements inside a list can be modified by assigning directly to an index. Elements can be added to the end using `append()`, inserted at specific indices using

`insert()`, or merged using `extend()`. Removal is performed using `remove(value)` or `pop(index)`.

Syntax Structure:

```
my_list.append(element)
my_list.insert(index, element)
my_list.remove(value)
removed_item = my_list.pop(index)
```

Example 1: Dynamic List Modification (Append & Insert):

```
fruits = ['apple', 'banana']
fruits.append('cherry')
print('After append:', fruits)
fruits.insert(1, 'mango')
print('After insert at index 1:', fruits)
fruits.extend(['orange', 'grape'])
print('After extend:', fruits)
```

Example 2: Removing Items (Remove & Pop):

```
numbers = [10, 20, 30, 40, 50]
numbers.remove(30) # Removes value 30
print('After remove(30):', numbers)
popped_item = numbers.pop(1) # Removes item at index 1
print('Popped Item:', popped_item)
print('Final List:', numbers)
```

Example 3: index ,count,copy:

Complete Code Example from Document

```
data = [10, 25, 30, 25, 40, 25, 50]

print('First index of 25:', data.index(25))

print('Occurrences of 25:', data.count(25))
```

```
backup_data = data.copy() # Creates a shallow copy
```

```
items = ['pen', 'pencil', 'eraser']
```

```
items.clear() # Clears all items from list
```

Graphical Working of List Methods (Mutable in Place)

Unlike strings, a list is mutable, so the SAME box row is edited in memory -- nothing new is created. `append()` simply adds one more box at the right end, as shown below:

Before append	apple	banana	
After append	apple	banana	cherry

The same before/after picture applies to every list-changing method used in this topic:

Operation	List Before	List After
<code>append('cherry')</code>	['apple', 'banana']	['apple', 'banana', 'cherry']
<code>insert(1, 'mango')</code>	['apple', 'banana', 'cherry']	['apple', 'mango', 'banana', 'cherry']
<code>extend(['orange', 'grape'])</code>	['apple', 'mango', 'banana', 'cherry']	[..., 'cherry', 'orange', 'grape']
<code>remove(30)</code>	[10, 20, 30, 40, 50]	[10, 20, 40, 50]
<code>pop(1) -> returns 20</code>	[10, 20, 40, 50]	[10, 40, 50]

Topic 4: List Searching, Sorting & Aggregation (sort, reverse, sum, min, max, len)

Lists can be sorted in ascending or descending order in place using `.sort()`. Python also provides built-in statistical aggregation utilities such as `len()`, `sum()`, `min()`, and `max()`.

Syntax Structure:

```
my_list.sort(reverse=False)
total_sum = sum(my_list)
average = sum(my_list) / len(my_list)
```

Example 1: Sorting and Reversing Lists:

```
scores = [45, 88, 12, 95, 60, 33]
scores.sort() # Ascending sort
print('Sorted Ascending:', scores)
scores.sort(reverse=True) # Descending sort
print('Sorted Descending:', scores)
scores.reverse() # Reverses current order
print('Reversed List:', scores)
```

Example 2: List Aggregation Operations:

```
marks = [75, 80, 92, 68, 85]
print('Total Marks:', sum(marks))
print('Maximum Mark:', max(marks))
print('Minimum Mark:', min(marks))
print('Number of Subjects:', len(marks))
print('Average Mark:', sum(marks) / len(marks))
```

Graphical Working of sort(), reverse() and Aggregation

sort() rearranges the SAME boxes into ascending order in place; reverse() simply flips the order of the boxes that already exist. Nothing is added or removed -- only the arrangement changes:

Operation	List Before	List After
sort()	[45, 88, 12, 95, 60, 33]	[12, 33, 45, 60, 88, 95]
sort(reverse=True)	[12, 33, 45, 60, 88, 95]	[95, 88, 60, 45, 33, 12]
reverse()	[95, 88, 60, 45, 33, 12]	[12, 33, 45, 60, 88, 95]

For marks = [75, 80, 92, 68, 85], len() counts the boxes (5), sum() adds every value inside the boxes (400), and min()/max() simply point at the smallest and largest box -- shown highlighted below (orange = min, green = max):

75	80	92	68	85
----	----	----	----	----

4. Part C: Tuples (tuple)

A Tuple is an immutable, ordered sequence of elements enclosed within parentheses `()`. Because tuples are immutable, their elements cannot be changed, added, or removed after creation. Tuples offer performance optimization, data protection against accidental modifications.

Topic 5: Tuple Operations & Tuple Unpacking

Tuples support standard indexing and slicing just like lists, but do not support item assignment. Tuple Unpacking allows assigning individual tuple elements directly to separate variables in a single clean line of code.

Syntax Structure:

tuple_name=()#empty tuple

tuple_name=(1,2,3)

Example 1: Tuple Access & Methods (count, index):

```
numbers = (10, 20, 30, 20, 40, 20)
print('Element at index 2:', numbers[2])
print('Count of 20:', numbers.count(20))
print('Index of 30:', numbers.index(30))
```

Example 2: Tuple Unpacking & Immutability Workaround:

```
student = (101, 'Krupa Pathak', 'BCA')
roll_no, name, course = student
print(f'Roll No: {roll_no}, Name: {name}, Course: {course}')
```

Workaround to modify a tuple: Convert to list -> Modify -> Convert back to tuple

```
tp = (1, 2, 3)
tp_list = list(tp)
```

```
tp_list.append(4)
tp = tuple(tp_list)
print('Modified Tuple via List conversion:', tp)
```

Graphical Working of Tuple Immutability & Unpacking

A tuple stores its items in numbered boxes exactly like a list, but the boxes are sealed the moment the tuple is created -- no item can be added, removed, or changed in place. The diagram below shows `student = (101, 'Krupa Pathak', 'BCA')`:

101	'Krupa Pathak'	'BCA'
<i>LOCKED (cannot edit)</i>	<i>LOCKED (cannot edit)</i>	<i>LOCKED (cannot edit)</i>

Tuple Unpacking simply copies each boxed value straight into a variable, in the same left-to-right order, with one line: `roll_no, name, course = student`

Variable	Value It Receives
roll_no	101
name	'Krupa Pathak'
course	'BCA'

Quick Recap: String vs List vs Tuple, Side by Side

Feature	String	List	Tuple
Symbol used	' ' or " "	[]	()
Mutable?	No	Yes	No
Ordered / Indexed?	Yes	Yes	Yes
Common methods	upper, lower, strip, split, replace	append, insert, remove, pop, sort	count, index
Typical use	Text data	Data that will change	Fixed data / dict keys